Chap 12. Presenting Data – Part 1

Sungwoo Tak

Networking Computing Lab

School of Computer Science and Engineering
Pusan National University

Standard file descriptors

- **STDIN**
  - The STDIN file descriptor references the standard input to the shell.
  - For a terminal interface, the standard input is the keyboard.
    - The shell receives input from the keyboard on the STDIN file descriptor, and processes each character as you type it.
- When you use the input redirect symbol (<), Linux replaces the standard input file descriptor with the file referenced by the redirection.
  - It reads the file and retrieves data just as if it were typed on the keyboard.
  - Many bash commands accept input from STDIN, especially if no files are specified on the command line. Here's an example of using the cat command with data entered from STDIN:

```
$ cat
this is a test
this is a test
this is a second test.
this is a second test.
```

- When you enter the cat command on the command line by itself, it accepts input from STDIN.
  - As you enter each line, the cat command echoes the line to the display.

Linux Standard File Descriptors

File Descriptor	Abbreviation	Description
0	STDIN	Standard input
1	STDOUT	Standard output
2	STDERR	Standard error

- However, you can also use the STDIN redirect symbol to force the cat command to accept input from another file other than STDIN:

```
$ cat < testfile
This is the first line.
This is the second line.
This is the third line.
$
```

- Now the cat command uses the lines that are contained in the testfile file as the input.
- You can use this technique to input data to any shell command that accepts data from STDIN.

STDOUT

- The STDOUT file descriptor references the standard output for the shell.
  - On a terminal interface, the standard output is the terminal monitor.
  - All output from the shell (including programs and scripts you run in the shell) is directed to the standard output, which is the monitor.
  - Most bash commands direct their output to the STDOUT file descriptor by default.

```
$ ls -l > test2
$ cat test2
total 20
-rw-rw-r-- 1 rich rich 53 2007-10-26 11:30 test
-rw-rw-r-- 1 rich rich  0 2007-10-26 11:32 test2
-rw-rw-r-- 1 rich rich 73 2007-10-26 11:23 testfile
$
```

You can also append data to a file. You do this using the >> symbol:

```
$ who >> test2
$ cat test2
total 20
-rw-rw-r-- 1 rich rich 53 2007-10-26 11:30 test
-rw-rw-r-- 1 rich rich  0 2007-10-26 11:32 test2
-rw-rw-r-- 1 rich rich 73 2007-10-26 11:23 testfile
rich     pts/0        2007-10-27 15:34 (192.168.1.2)
$
```

STDERR

- The shell handles error messages using the special STDERR file descriptor.
  - The STDERR file descriptor references the standard error output for the shell.
  - This is the location where the shell sends error messages generated by the shell or programs and scripts running in the shell.
- By default, the STDERR file descriptor points to the same place as the STDOUT file descriptor (even though they are assigned different file descriptor values).
  - This means that, by default, all error messages go to the monitor display.
- However, as you saw in the example, when you redirect STDOUT this doesn't automatically redirect STDERR.
- When working with scripts, you'll often want to change that behavior, especially if you're interested in logging error messages to a log file.

STDERR: Redirecting just errors


```
$ ls -al badfile 2> test4
$ cat test4
ls: cannot access badfile: No such file or directory
$
```

```
$ ls -al test badtest test2 2> test5
-rw-rw-r-- 1 rich rich 158 2007-10-26 11:32 test2
$ cat test5
ls: cannot access test: No such file or directory
ls: cannot access badtest: No such file or directory
$
```

Redirecting errors and data


```
$ ls -al test test2 test3 badtest 2> test6 1> test7
$ cat test6
ls: cannot access test: No such file or directory
ls: cannot access badtest: No such file or directory
$ cat test7
-rw-rw-r-- 1 rich rich 158 2007-10-26 11:32 test2
-rw-rw-r-- 1 rich rich   0 2007-10-26 11:33 test3
$
```

- The shell redirects the normal output of the ls command that would have gone to STDOUT to the test7 file using the 1> symbol.
- Any error messages that would have gone to STDERR were redirected to the test6 file using the 2> symbol.

Redirecting errors and data (continued..)


```
$ ls -al test test2 test3 badtest &> test7
$ cat test7
ls: cannot access test: No such file or directory
ls: cannot access badtest: No such file or directory
-rw-rw-r-- 1 rich rich 158 2007-10-26 11:32 test2
-rw-rw-r-- 1 rich rich   0 2007-10-26 11:33 test3
$
```

- Using the &> symbol, all of the output generated by the command is sent to the same location, both data and errors.
- You'll notice that one of the error messages is out of order from what you'd expect.
  - The error message for the badtest file (the last file to be listed) appears second in the output file.
  - The bash shell automatically gives error messages a higher priority than the standard output.
  - This allows you to view the error messages together, rather than scattered throughout the output file.

Redirecting Output in Scripts

- When you redirect to a file descriptor, you must precede the file descriptor number with an ampersand sign (&):

  echo "This is an error message" >&2

- This line displays the text wherever the STDERR file descriptor for the script is pointing, instead of the normal STDOUT.
- Here's an example of a script that uses this feature:

```
$ cat test8
#!/bin/bash
# testing STDERR messages

echo "This is an error" >&2
echo "This is normal output"
$
```

```
$ ./test8
This is an error
This is normal output
$
```

Remember, by default Linux directs the STDERR output to STDOUT. However, if you redirect STDERR when running the script, any text directed to STDERR in the script will be redirected:

```
$ ./test8 2> test9
This is normal output
```

```
$ cat test9
This is an error
$
```

Permanent redirections

- If you have lots of data that you're redirecting in your script, it can get tedious having to redirect every echo statement.
  - Instead, you can tell the shell to redirect a specific file descriptor for the duration of the script by using the exec command:

```
$ cat test10
#!/bin/bash
# redirecting all output to a file
exec 1>testout

echo "This is a test of redirecting all output"
echo "from a script to another file."
echo "without having to redirect every individual line"
$ ./test10
$ cat testout
This is a test of redirecting all output
from a script to another file.
without having to redirect every individual line
$
```

- The exec command starts a new shell, and redirects the STDOUT file descriptor to a file.
  - All output in the script that goes to STDOUT is instead redirected to the file.

Permanent redirections (continued..)


```
$ cat test11
#!/bin/bash
# redirecting output to different locations

exec 2>testerror

echo "This is the start of the script"
echo "now reidirecting all output to another location"

exec 1>testout

echo "This output should go to the testout file"
echo "but this should go to the testerror file" >&2
$ ./test11
This is the start of the script
now reidirecting all output to another location
$ cat testout
This output should go to the testout file
$ cat testerror
but this should go to the testerror file
$
```

Redirecting Input in Scripts

- You can use the same technique used to redirect STDOUT and STDERR in your scripts to redirect STDIN from the keyboard.
  - The exec command allows you to redirect STDIN to a file on the Linux system:

    exec 0 < testfile
- This command informs the shell that it should retrieve input from the file testfile instead of STDIN.

```
$ cat test12
#!/bin/bash
# redirecting file input

exec 0< testfile
count=1

while read line
do
   echo "Line #$count: $line"
   count=$[ $count + 1 ]
done
```

```
$ ./test12
Line #1: This is the first line.
Line #2: This is the second line.
Line #3: This is the third line.
$
```